



UNIVERSITI TEKNIKAL MALAYSIA MELAKA

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

WORKSHOP 1

REPORT

Name : MUHAMMAD UZAIR BIN SHAMSULBAHARIN

Matric Number : B032410498

Program : BITM SIG2

Project Title : HOUSE RENTAL MANAGEMENT SYSTEM

Supervisor Name : DR. MOHAMAD LUTFI BIN DOLHALIT

Evaluator Name : NAZREEN BIN ABDULLASIM

EXECUTIVE SUMMARY

The House Rental Management System (HRMS) is a console based application developed in C++ to centralize and streamline the management of rental properties. The system successfully addresses the core needs of three distinct user roles: Landlords, Property Representatives, and Tenants. It provides functionalities for user registration and authentication, property and tenant management, financial tracking, booking request handling, and automated report generation. The primary goal is to replace manual, error prone record keeping with an efficient, centralized digital solution.

The application is a console-based program where responsibilities are clearly defined, but the components are closely connected. The user interface logic is handled separately from the data access and business logic, which directly interacts with a MySQL database.

This analysis finds the system to be functional for its intended scope. However, it also identifies critical areas for improvement. The most severe issue is the presence of significant SQL vulnerabilities, which pose a major security risk. Other concerns include a lack of robust error handling for user input, the use of hardcoded database credentials, and the absence of a proper data access layer, leading to duplicated code and poor maintainability.

In conclusion, HRMS is a functional proof-of-concept that demonstrates a solid understanding of the problem domain. To evolve into a production ready application, future efforts must prioritize security hardening by implementing prepared statements universally, enhancing input validation, and improving the overall software to increase modularity and maintainability.

TABLE OF CONTENTS

	PAGE
EXECUTIVE SUMMARY	1
TABLE OF CONTENTS	2
LIST OF TABLES ← (header 4)	3
LIST OF FIGURES ← (header 5)	4
CHAPTER 1: INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Objective(s) of the project	1
1.4 Scope	2
1.4.1 Modules to be developed	2
1.4.2 Target User	3
1.5 Project Significance	3
1.6 Gantt Chart of Project Activities	4
CHAPTER 2: ANALYSIS OF PROBLEM	5
2.1 Problem Decomposition Description	5
2.2 Structured Chart	6
CHAPTER 3: DESIGN	7
3.1 Business Rules	7
3.2 Flowchart	8
3.3 ERD	13
3.4 Data Normalization	14
3.4.1 Step 1: First Normal Form (1NF)	14
3.4.2 Step 2: Second Normal Form (2NF)	15
3.4.3 Step 3: Third Normal Form (3NF)	16
3.5 Data Dictionary	17
3.6 Interface Design	21
CHAPTER 4: IMPLEMENTATION	24
4.1 Naming Convention	24
4.2 Function	25
4.3 Array	26
4.4 Selection	28
4.5 Control	29
4.6 Pointer	30
4.7 Error Handling	30
4.8 Business Rule Implementation	33
4.9 Complex Calculation Implementation	34
4.10 Report Generation for Analysis Implementation	36
CHAPTER 5: CONCLUSION	39

	3
5.1 Constraints	39
5.2 Future Improvements	39
REFERENCES	41

LIST OF TABLES

	PAGE
<i>Table 3.4.1.1: List of all existing columns in First Normal Form (1NF).....</i>	<i>14</i>
<i>Table 3.4.1.1: Tenancies Table (1NF).....</i>	<i>14</i>
<i>Table 3.4.1.2: Payments Table (1NF).....</i>	<i>15</i>
<i>Table 3.4.2.1: Landlords Table (2NF).....</i>	<i>15</i>
<i>Table 3.4.2.2: Houses Table (2NF).....</i>	<i>15</i>
<i>Table 3.4.2.3: Tenants Table (2NF).....</i>	<i>16</i>
<i>Table 3.4.2.4: Tenancies Table (2NF).....</i>	<i>16</i>
<i>Table 3.5.1: Data Dictionary for Landlord.....</i>	<i>17</i>
<i>Table 3.5.2: Data Dictionary for House.....</i>	<i>17</i>
<i>Table 3.5.3: Data Dictionary for Tenant.....</i>	<i>18</i>
<i>Table 3.5.4: Data Dictionary for Representative.....</i>	<i>19</i>
<i>Table 3.5.5: Data Dictionary for Payment.....</i>	<i>19</i>
<i>Table 3.5.6: Data Dictionary for Booking Request.....</i>	<i>20</i>
<i>Table 4.1.1: Naming Convention Examples.....</i>	<i>24</i>

LIST OF FIGURES

	PAGE
<i>Figure 1.6.1: Gantt Chart of Project Activities.....</i>	4
<i>Figure 2.2.1: HRMS Structure Chart.....</i>	6
<i>Figure 3.2.1: Main Menu Flowchart.....</i>	8
<i>Figure 3.2.2: Tenant Menu Flowchart.....</i>	9
<i>Figure 3.2.3: Tenant Functions Flowchart.....</i>	10
<i>Figure 3.2.4: Representative Functions Flowchart.....</i>	11
<i>Figure 3.2.4: Landlord Functions Flowchart.....</i>	12
<i>Figure 3.3.1: Entity Relationship Diagram (ERD).....</i>	13
<i>Figure 3.6.1: Landlord Main Menu Interface.....</i>	21
<i>Figure 3.6.2: Representative Main Menu Interface.....</i>	22
<i>Figure 3.6.3: Tenant Main Menu Interface.....</i>	22
<i>Figure 3.6.4: Example of Successful message feedback.....</i>	23
<i>Figure 3.6.4: Example of Error message feedback.....</i>	23
<i>Figure 4.2.1: Example of Function Prototypes in db_cruds.h.....</i>	25
<i>Figure 4.2.2: Example of Function Prototypes in misc_functions.h.....</i>	26
<i>Figure 4.3.1: Example of std::vector use in arrowMenu() function.....</i>	27
<i>Figure 4.3.1: Example of std::vector use in reviewPendingRequestsUI() function.....</i>	27
<i>Figure 4.4.1: Example of switch case in LandlordMenu() function.....</i>	28
<i>Figure 4.4.1: Example of if-else statement in deleteHouse() function.....</i>	28
<i>Figure 4.5.1: Example of do-while loops in LandlordMenu() function.....</i>	29
<i>Figure 4.5.2: Example of while loops in listTenants() function.....</i>	29
<i>Figure 4.6.1: Example of unique_ptr in updateHouseOccupancy() function.....</i>	30
<i>Figure 4.6.2: Example of sql::Connection* con.....</i>	30
<i>Figure 4.7.1: Example of try-catch for database error handling.....</i>	31
<i>Figure 4.7.2: Example of error checking for empty input.....</i>	31
<i>Figure 4.7.3: Example of validating input for month in searchBookHouse() function.....</i>	31
<i>Figure 4.8.1: Example of vacancy check before tenant can book a house.....</i>	33
<i>Figure 4.8.2: Example of permission check for destructive actions.....</i>	33
<i>Figure 4.9.1: Calculation for overpaid and forwarding to following month.....</i>	34
<i>Figure 4.9.2: Complex calculation of tenant's payment status.....</i>	35
<i>Figure 4.9.3: Example of complex SQL query for financial data of every property.....</i>	35
<i>Figure 4.10.1: Example output for financial summary report.....</i>	36
<i>Figure 4.10.2: Example output for property overview report.....</i>	37
<i>Figure 4.10.3: Example of exporting and viewing the full report.....</i>	38

CHAPTER 1: INTRODUCTION

1.1 Introduction

The **House Rental Management System (HRMS)** is a C++ application designed to manage the operational complexities of the property rental business. It serves as a centralized platform for landlords, their representatives, and tenants to interact with property data, manage bookings, and track financial records. By moving these processes from manual methods to a structured software system, HRMS aims to enhance efficiency, reduce administrative overhead, and improve data accuracy and accessibility for all stakeholders. The system is built as a console application that interfaces with a MySQL database for data persistence.

1.2 Problem Statement

Managing rental properties manually is an inefficient, error prone, and time consuming process. Landlords and representatives often struggle with disorganized data spread across multiple documents, leading to difficulties in tracking tenant information, payment statuses, and property availability. This lack of a centralized system results in delayed communication, inaccurate financial reporting, and a poor experience for tenants. The HRMS project directly addresses these challenges by providing a unified digital platform that automates record keeping, simplifies user interactions, and provides instant access to critical data, thereby enhancing operational efficiency and data accuracy.

1.3 Objective(s) of the project

This project embarks on the following objectives:

1. **To implement four modular functionalities with CRUDS (Create, Read, Update, Delete, Search) database management** for the House Rental Management System:

- **User Management Module:** To manage the registration, authentication, and profile data for Landlords, Representatives, and Tenants.
 - **Property Management Module:** To handle the creation, viewing, updating, and deletion of house listings, including details on rent, vacancy, and assigned representatives.
 - **Booking & Tenancy Module:** To facilitate tenant-initiated booking requests, review and approval/rejection by authorized users, and automated updates to tenancy statuses.
 - **Financial Management Module:** To record, edit, and delete payments, track tenant payment histories, and generate payment receipts.
2. To implement complex calculations for financial and occupancy analysis, including tenant credit balance, remaining dues on rent, property occupancy rates, and potential revenue loss from vacancies.
 3. To develop comprehensive summary and detailed analysis reports for the House Rental Management System, including financial summaries, outstanding payment reports, property occupancy analyses, and full system reports exported to text files.

1.4 Scope

1.4.1 Modules to be developed

The system is composed of several interconnected modules, each providing a specific set of functionalities:

- A. **Authentication and User Management:** Secure login and registration for Landlords, Representatives, and Tenants. Users can manage their own profile details, including changing passwords and personal information.
- B. **Landlord Module:** Landlords can perform CRUD operations on their properties, assign representatives, and access a suite of analytical reports covering revenue, occupancy, and outstanding payments.

- C. **Representative Module:** Representatives manage houses and tenants assigned to them, review and process booking requests, and manage payment records for tenants in their managed properties.
- D. **Tenant Module:** Tenants can browse and search for available houses, submit booking requests for a specified duration, view their booking history, and track their payment records.
- E. **Reporting Module:** Generates reports for landlords and representatives, as well as printable text file receipts for individual payment transactions.

1.4.2 Target User

The system is designed for three distinct roles:

1. **Landlords:** Property owners who need a high level overview of their assets, financial performance, and management efficiency.
2. **Representatives:** Property managers tasked with the day to day operations of managing houses, interacting with tenants, and handling payments.
3. **Tenants:** Renters seeking to find a property, manage their tenancy, and keep track of their financial obligations.

1.5 Project Significance

The HRMS project holds significant value by providing an accessible and efficient tool for small-scale rental management. It centralizes data, reduces manual administrative workload, and minimizes human error. For landlords, it offers powerful insights into their investment performance. For representatives, it streamlines daily tasks and improves tenant relations. For tenants, it provides a transparent and convenient way to interact with property managers. To summarize, the system gives a more organized, transparent, and professional rental ecosystem for all parties involved.

1.6 Gantt Chart of Project Activities

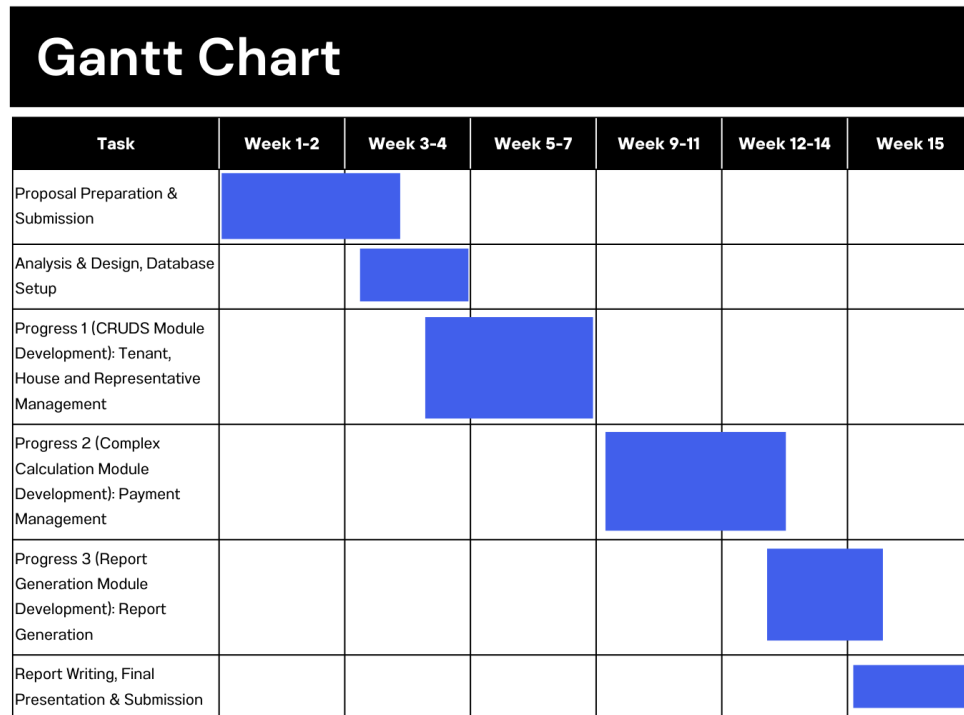


Figure 1.6.1: Gantt Chart of Project Activities

CHAPTER 2: ANALYSIS OF PROBLEM

2.1 Problem Decomposition Description

The primary problem of managing a house rental system is decomposed into five problems and sub-problems, each corresponding to a core domain of the system.

1. **User Management:** This involves handling the profile of all types of users. It includes creating new accounts for landlords, representatives, and tenants, authenticating users at login, and allowing users to view and update their own profile information.
2. **Property Management:** This focuses on the physical assets (houses). It requires functionalities for landlords to add new properties, update details of existing ones (e.g., rent, assigned representative), view a list of all their properties, and delete properties.
3. **Tenancy and Booking Management:** This sub-problem addresses the relationship between tenants and properties. It includes the process for tenants to search for houses and submit booking requests. It also involves the logic for representatives or landlords to review these requests, approve or reject them, and the will automatically update the tenant's status (e.g., from 'Pending' to 'Active').
4. **Financial Management:** This covers all payment transactions. It includes recording rent payments from tenants, updating payment statuses (e.g., 'Paid', 'Overdue'), calculating balances, processing refunds, and allowing users to view payment histories.
5. **Reporting and Analytics:** This involves aggregating raw data into meaningful information. The system needs to generate financial summaries for landlords, list outstanding payments, analyze property occupancy rates, and produce individual payment receipts for record keeping.

2.2 Structured Chart

The following chart illustrates the high-level functional hierarchy and control flow of the application, starting from the main entry point.

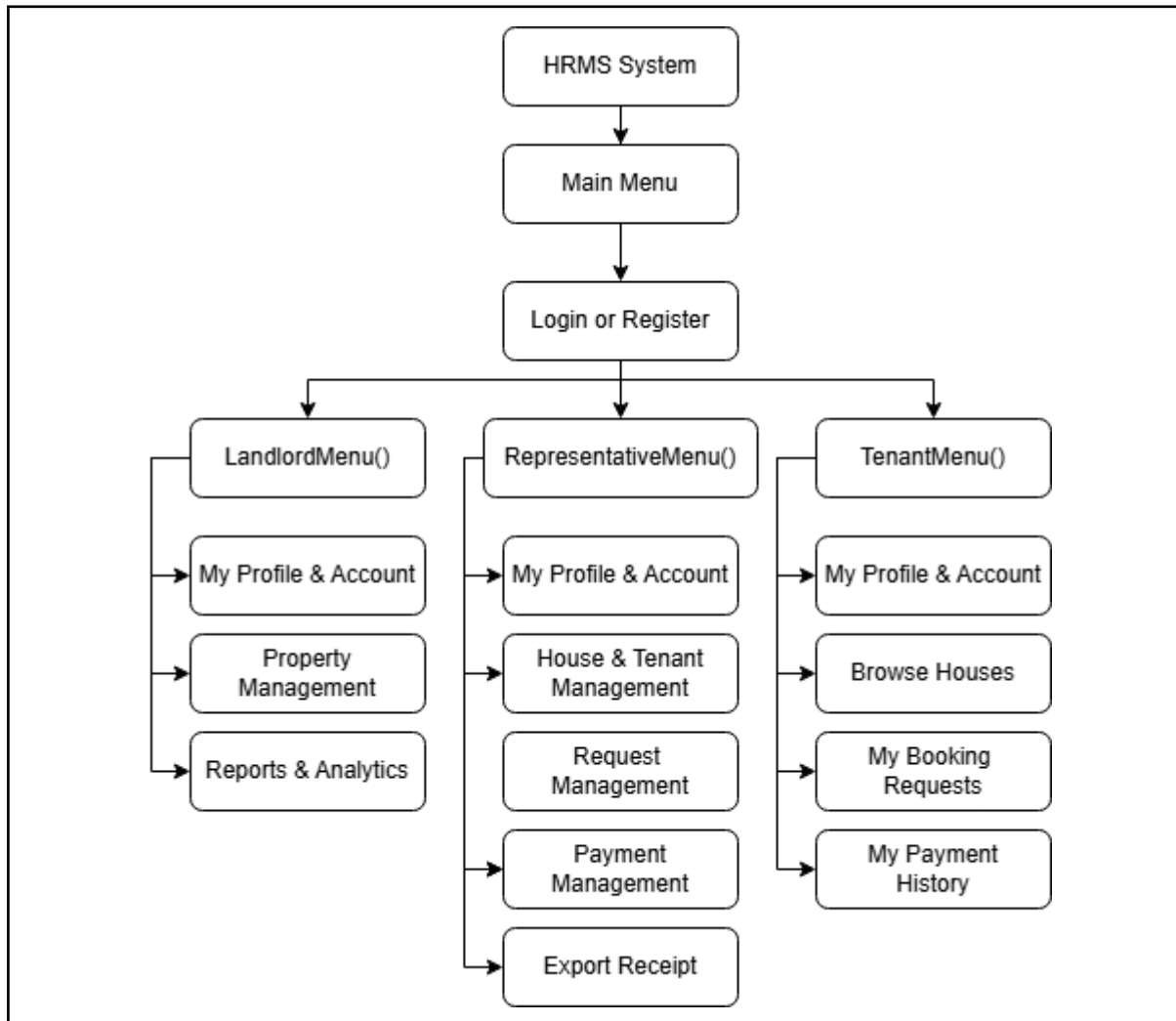


Figure 2.2.1: HRMS Structure Chart

CHAPTER 3: DESIGN

3.1 Business Rules

The system operates on a set of implicit and explicit business rules derived from the source code:

1. A user must register as a **Landlord**, **Representative**, or **Tenant**.
2. Only landlords can create, update, or delete their own houses.
3. A house must have a vacancy limit, which cannot be set below its current occupancy.
4. A tenant can search for available houses and submit a booking request for a specific start month and duration.
5. A booking request is 'Pending' until reviewed by a representative or landlord.
6. A booking request can only be approved if the house has available slots and the tenant does not have another overlapping tenancy.
7. Approving a booking request automatically updates the tenant's status to 'Active' and assigns them to the house.
8. Tenancy statuses ('Pending', 'Active', 'Completed', 'Moved Out', 'Cancelled') are automatically computed based on dates and recorded actions.
9. Payments can be marked as 'Paid', 'Partial', 'Overdue', or 'Refunded'.
10. Tenants with a positive credit balance upon moving out are automatically issued a refund record.
11. Users can only update their own profile information (name, phone, username, password).

3.2 Flowchart

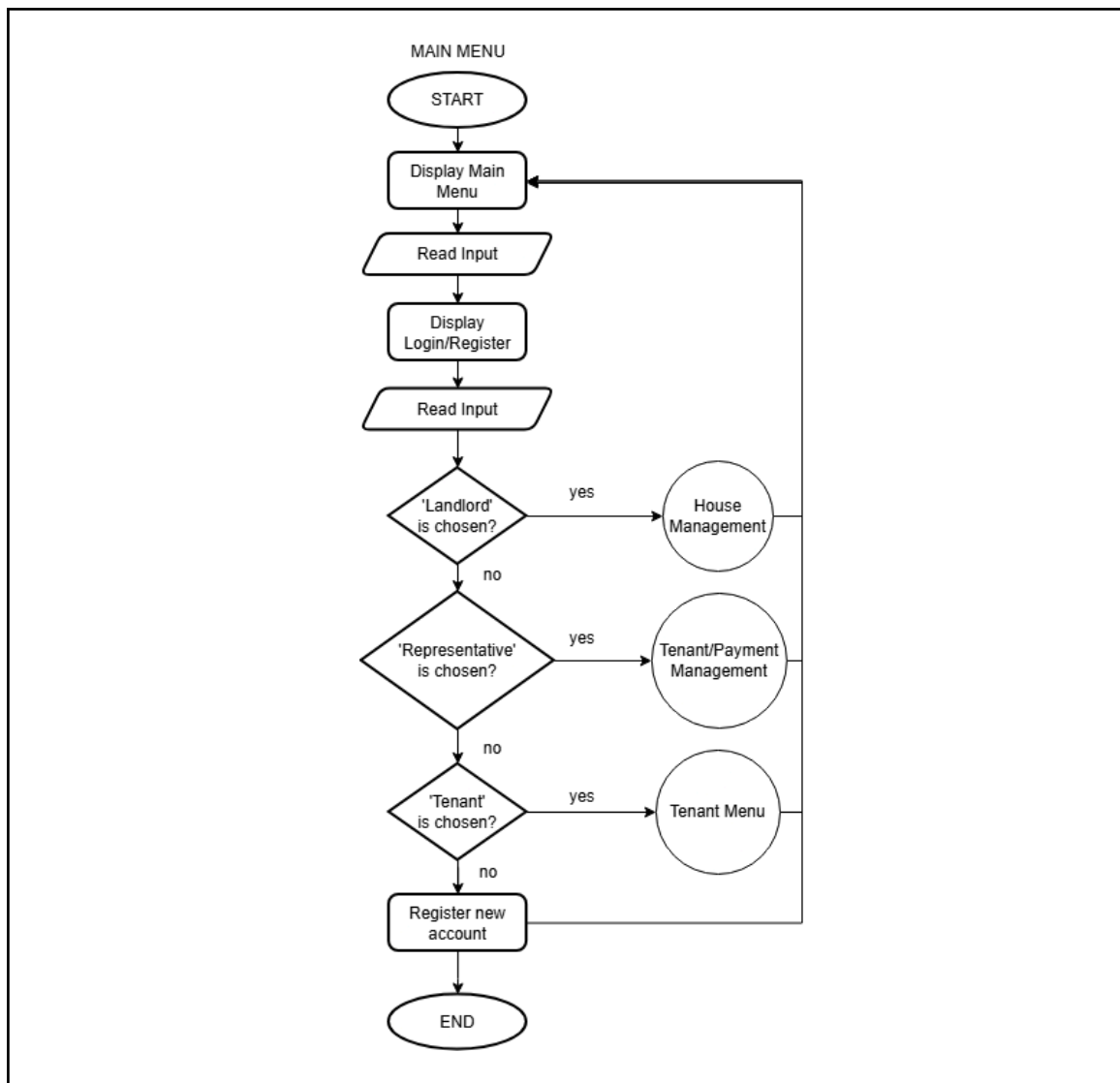


Figure 3.2.1: Main Menu Flowchart

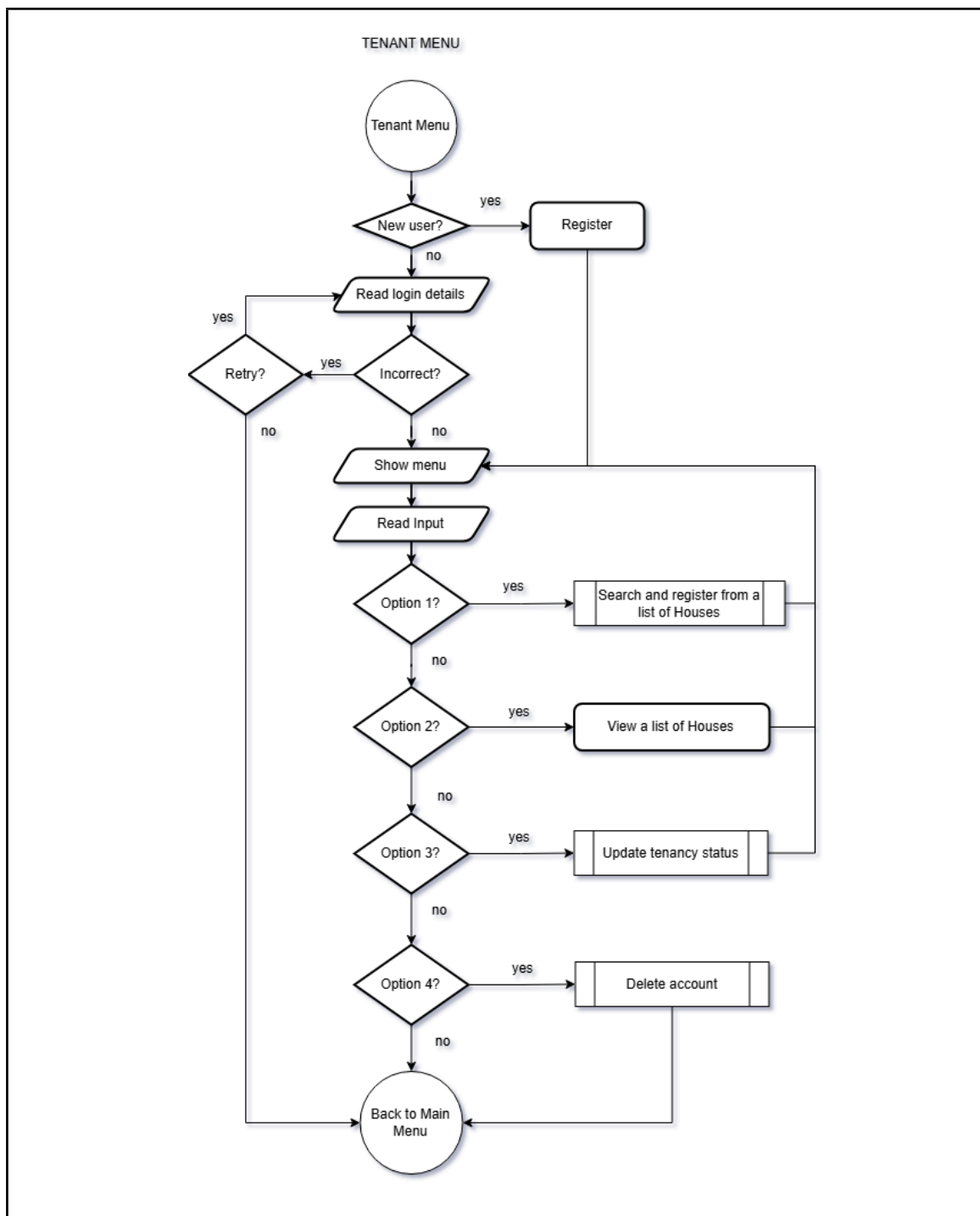


Figure 3.2.2: Tenant Menu Flowchart

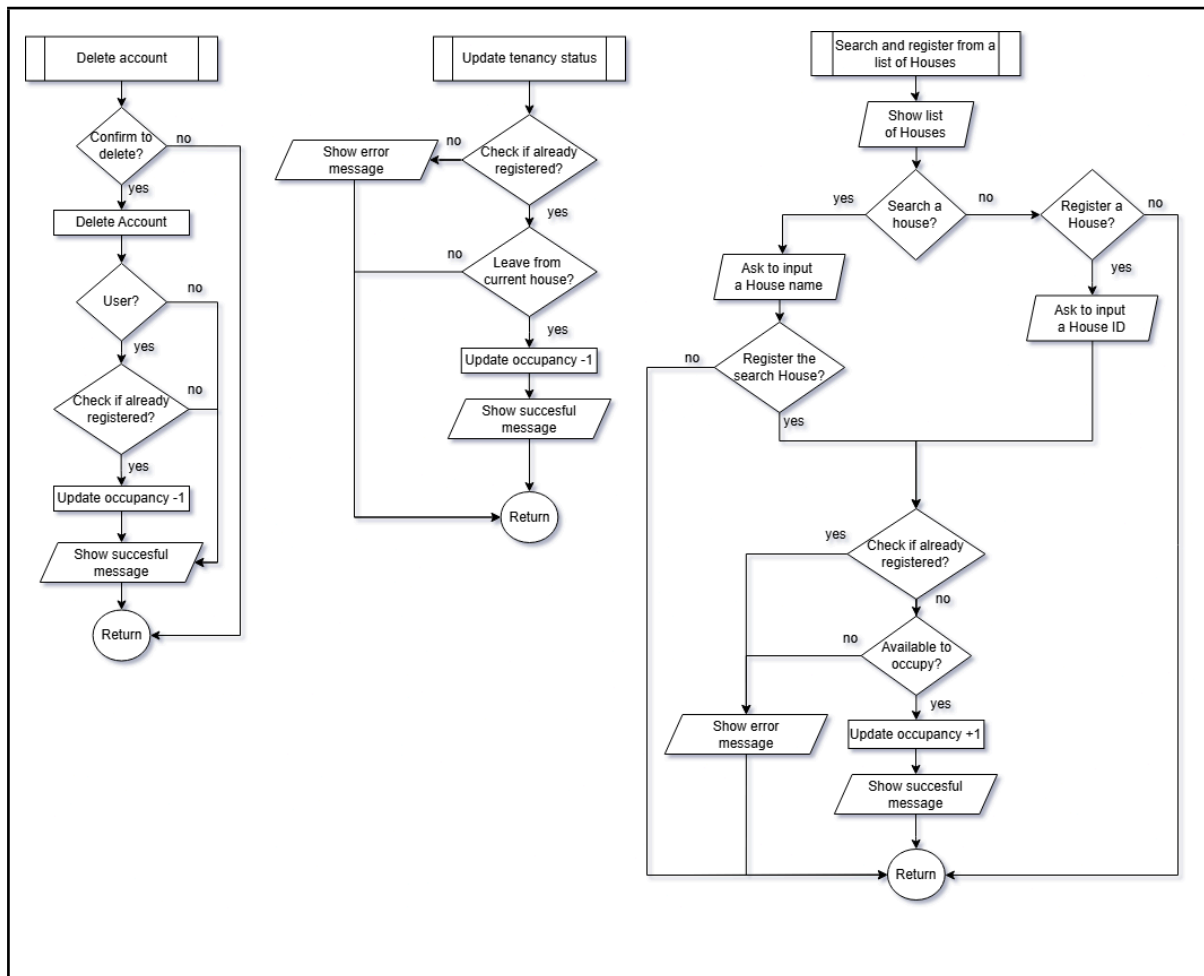


Figure 3.2.3: Tenant Functions Flowchart

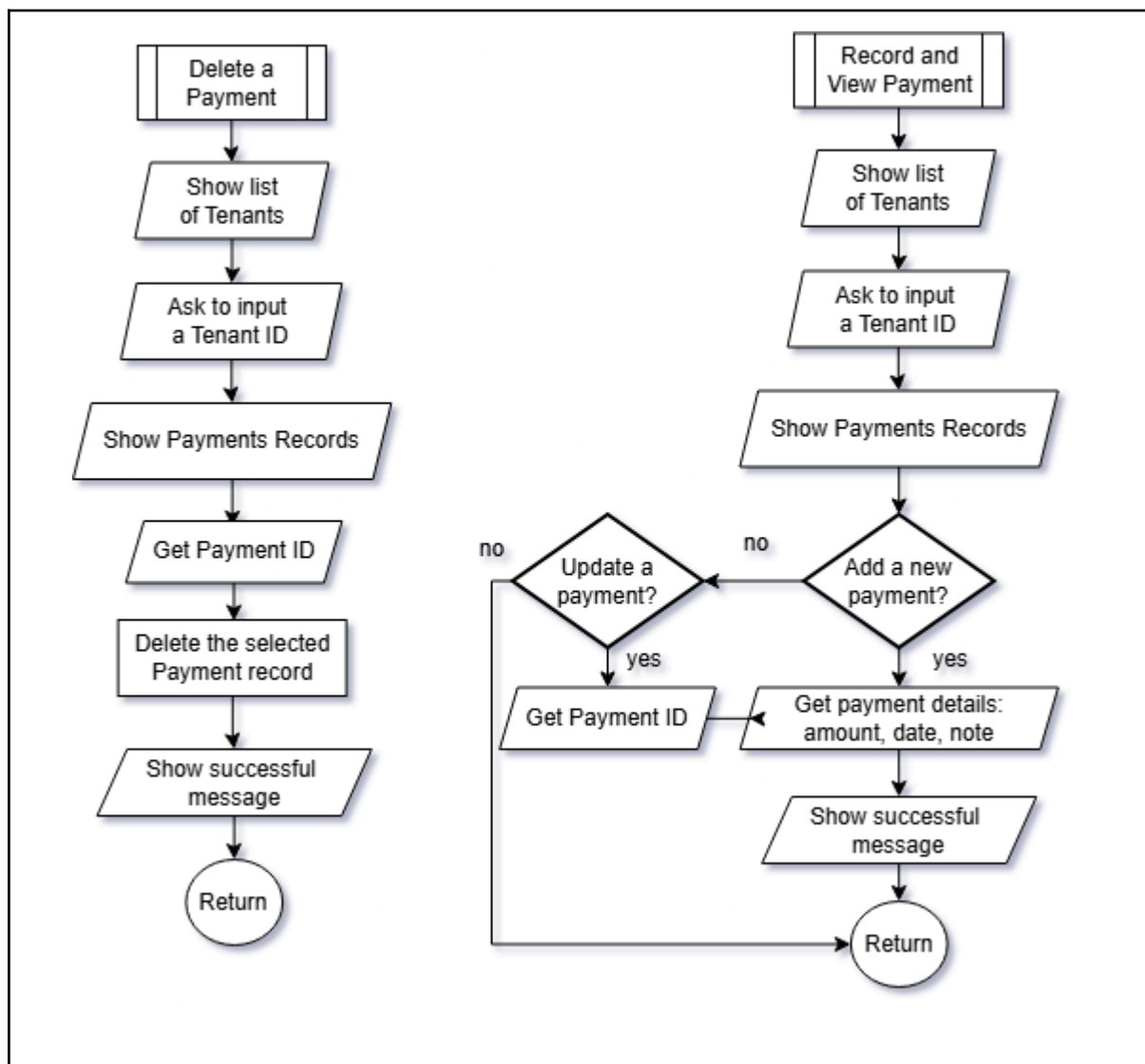


Figure 3.2.4: Representative Functions Flowchart

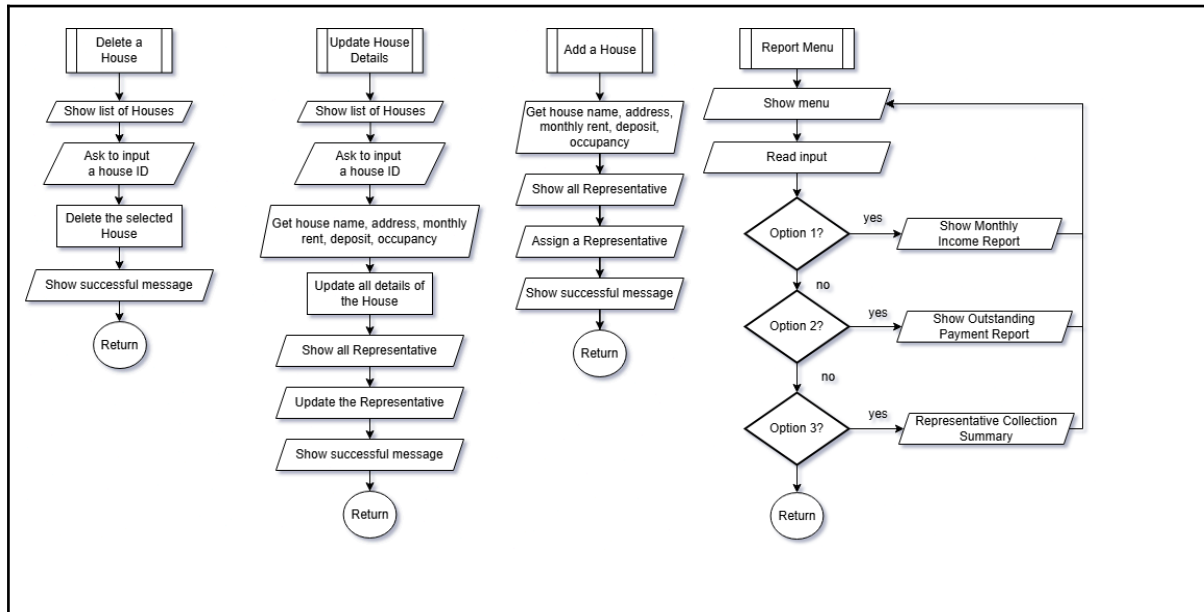


Figure 3.2.4: Landlord Functions Flowchart

3.3 ERD

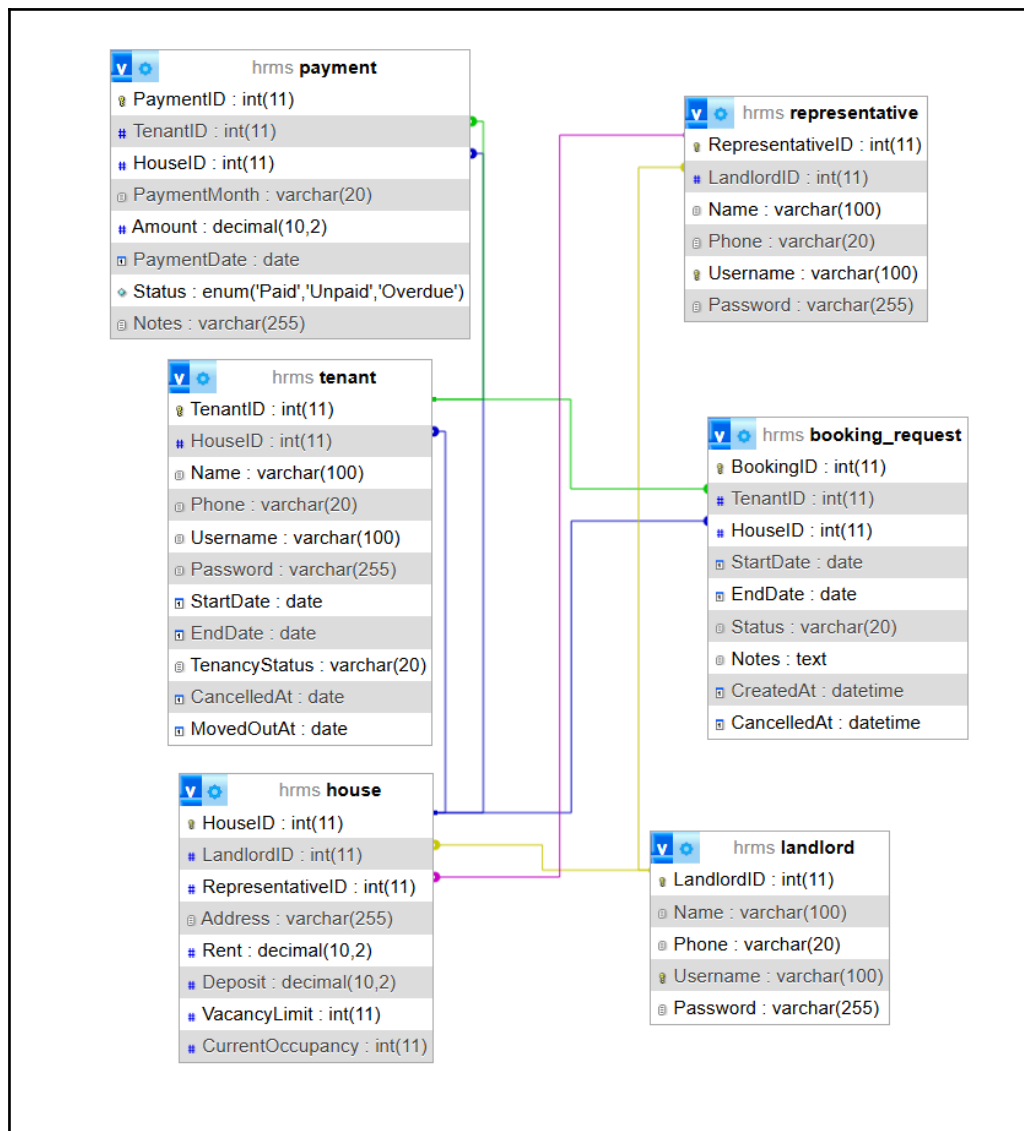


Figure 3.3.1: Entity Relationship Diagram (ERD)

3.4 Data Normalization

3.4.1 Step 1: First Normal Form (1NF)

Table 3.4.1.1: List of all existing columns in First Normal Form (1NF)

House_A ddress	Landlor d_Name	Landlor d_Phone	Tenant_ Name	Tenant_ Phone	Tenancy _Start
Rent	Payment1_ Date	Payment1_ Amount	Payment2_ Date	Payment2_ Amount	

Rule:

1. Ensure the table has a Primary Key to uniquely identify each row.
2. Eliminate repeating groups. Each column must hold a single, atomic value.

To achieve 1NF, solve repeating payments problems by creating a separate `Payments` table.

The Fix:

1. Break the single table into two: `Tenancies` and `Payments`.
2. Each table gets a primary key (`TenancyID`, `PaymentID`).
3. The `Payments` table uses a Foreign Key (`TenancyID`) to link back to the specific tenancy it belongs to.

Resulting Tables (Now in 1NF):

Table 3.4.1.1: Tenancies Table (1NF)

Table Tenancies							
Tena ncyI D (PK)	Hous e_Ad dres s	Land lord _Nam e	Land lord _Pho ne	Tena nt_N ame	Tena nt_P hone	Tena ncy_ Star t	Rent

Table 3.4.1.2: Payments Table (1NF)

Table Payments			
PaymentID (PK)	TenancyID (FK)	Payment_Date	Payment_Amount

3.4.2 Step 2: Second Normal Form (2NF)

Rule:

1. Must be in 1NF.
2. All non key attributes must be fully dependent on the entire primary key. (This rule is most relevant for tables with composite primary keys, but the principle helps us identify misplaced data).

Separate entities that don't depend on the TenancyID. A House and its Landlord exist independently of any tenancy.

The Fix:

1. Create a Houses table to store property information.
2. Create a Landlords table.
3. Create a Tenants table.
4. The Tenancies table becomes a "linking table" that just connects a Tenant to a House.

Resulting Tables (Now in 2NF):

Table 3.4.2.1: Landlords Table (2NF)

Table Landlords		
LandlordID (PK)	Landlord_Name	Landlord_Phone

Table 3.4.2.2: Houses Table (2NF)

Table Houses

HouseID (PK)	House_Addres s	Rent	LandlordID (FK)
--------------	-------------------	------	--------------------

Table 3.4.2.3: Tenants Table (2NF)

Table Tenants		
TenantID (PK)	Tenant_Name	Tenant_Phone

Table 3.4.2.4: Tenancies Table (2NF)

Table Tenancies			
TenancyID (PK)	HouseID (FK)	TenantID (FK)	Tenancy_Star t

3.4.3 Step 3: Third Normal Form (3NF)

Rule:

1. Must be in 2NF.
2. There must be no **transitive dependencies**. A transitive dependency is when a non key attribute depends on another non key attribute, instead of depending on the primary key.

The 2NF design is already very close to 3NF because it correctly separates the entities. Here, `Landlord_Name` does not depend on `HouseID` (the primary key). It depends on `LandlordID` (a non key attribute). This is a transitive dependency.

The Fix:

1. Remove `Landlord_Name` from the `Houses` table.
2. To find the landlord's name for a house, perform a `JOIN` between the `Houses` and `Landlords` tables using `LandlordID`.

Thus, the final Result (3NF Design of HRMS):

- **landlord Table:** Stores only landlord information.
- **house Table:** Stores only house information, using LandlordID as a foreign key.
- **tenant Table:** Stores only tenant information.
- **payment Table:** Stores only payment information, using TenantID and HouseID as foreign keys.
- **representative Table:** Stores only representative information.
- **Booking_request Table:** An event where a tenant asks to rent a house.

3.5 Data Dictionary

Table 3.5.1: Data Dictionary for Landlord

Landlord Table			
Attribute	Data Type	Constraints / Key	Description / Relationship
LandlordID	INT	PK, AI	Unique identifier for each landlord. (AI: Auto-Increment)
name	VARCHAR (255)	NOT NULL	The full name of the landlord.
phone	VARCHAR (20)	NULL	The landlord's contact phone number.
username	VARCHAR (50)	NOT NULL, UNIQUE	The unique username used for logging in.
password	VARCHAR (255)	NOT NULL	The login password.

Table 3.5.2: Data Dictionary for House

House Table			
Attribute	Data Type	Constraints / Key	Description / Relationship
HouseID	INT	PK, AI	Unique identifier for each house.

LandlordID	INT	FK, NOT NULL	References landlord(LandlordID). The owner of the property.
RepresentativeID	INT	FK, NULL	References representative(RepresentativeID). The manager of the property.
address	VARCHAR(255)	NOT NULL	The physical street address of the property.
rent	DECIMAL(10,2)	NOT NULL	The monthly rental amount.
deposit	DECIMAL(10,2)	NOT NULL	The security deposit required from a tenant.
VacancyLimit	INT	NOT NULL, DEFAULT 1	The maximum number of tenants allowed in the house.
CurrentOccupancy	INT	NOT NULL, DEFAULT 0	The current number of tenants with an active or pending tenancy.

Table 3.5.3: Data Dictionary for Tenant

Tenant Table			
Attribute	Data Type	Constraints / Key	Description / Relationship
TenantID	INT	PK, AI	Unique identifier for each tenant.
HouseID	INT	FK, NULL	References house(HouseID). The house the tenant is currently assigned to.
name	VARCHAR(255)	NOT NULL	The full name of the tenant.
phone	VARCHAR(20)	NULL	The tenant's contact phone number.
username	VARCHAR(50)	NOT NULL, UNIQUE	The unique username used for logging in.
password	VARCHAR(255)	NOT NULL	The login password. (Note: Should be hashed in

			production.)
StartDate	DATE	NULL	The start date of the current or upcoming tenancy.
EndDate	DATE	NULL	The end date of the current tenancy.
TenancyStatus	VARCHAR(20)	NOT NULL	The tenant's current status (e.g., 'Active', 'Pending', 'Completed').
MovedOutAt	DATETIME	NULL	Timestamp for when a tenant moves out before the EndDate.
CancelledAt	DATETIME	NULL	Timestamp for when a tenancy is administratively cancelled.

Table 3.5.4: Data Dictionary for Representative

Representative Table			
Attribute	Data Type	Constraints / Key	Description / Relationship
RepresentativeID	INT	PK, AI	Unique identifier for each representative.
name	VARCHAR (255)	NOT NULL	The full name of the representative.
phone	VARCHAR (20)	NULL	The representative's contact phone number.
username	VARCHAR (50)	NOT NULL, UNIQUE	The unique username used for logging in.
password	VARCHAR (255)	NOT NULL	The login password.

Table 3.5.5: Data Dictionary for Payment

Payment Table			
Attribute (Column Name)	Data Type	Constraints / Key	Description / Relationship
PaymentID	INT	PK, AI	Unique identifier for the

			payment transaction.
TenantID	INT	FK, NOT NULL	References tenant(TenantID). The tenant associated with the payment.
HouseID	INT	FK, NOT NULL	References house(HouseID). The house the payment is for.
Amount	DECIMAL(10,2)	NOT NULL	The amount paid. A negative value indicates a refund.
PaymentDate	DATE	NOT NULL	The date the payment was made.
Status	VARCHAR(20)	NOT NULL	The status of the payment ('Paid', 'Partial', 'Overdue', 'Refunded').
Notes	TEXT	NULL	Optional notes related to the transaction.

Table 3.5.6: Data Dictionary for Booking Request

Booking Request Table			
Attribute	Data Type	Constraints / Key	Description / Relationship
BookingID	INT	PK, AI	Unique identifier for the booking request.
TenantID	INT	FK, NOT NULL	References tenant(TenantID). The tenant making the request.
HouseID	INT	FK, NOT NULL	References house(HouseID). The house being requested.
StartDate	DATE	NOT NULL	The proposed start date of the tenancy.
EndDate	DATE	NOT NULL	The proposed end date of the tenancy.
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending'	The status of the request ('Pending', 'Approved', 'Rejected').

3.6 Interface Design

The user interface is text-based and designed for a standard console window. It relies on a helper library (`ui_helpers.h`) to create a consistent and user-friendly experience using ASCII characters.

- **Layout:** Menus and prompts are centered horizontally. A boxed design with borders is used for menus to clearly separate options from other text.
- **Navigation:** Arrow keys (Up/Down) are used for selection, and Enter confirms a choice. This is a significant improvement over simple numeric input, as it feels more interactive and reduces errors.
- **Color Coding:** The interface uses color to convey information and improve readability:
 - **Blue/Cyan:** Titles and headers.
 - **Green:** Success messages, positive financial numbers.
 - **Yellow:** Informational text and prompts.
 - **Red:** Warnings and error messages.
 - **Magenta:** Highlighted menu selections.
- **Feedback:** The system provides immediate feedback for actions, such as "House added successfully!" or "Invalid selection." Loading animations are used for operations that may take a moment, like initialization.

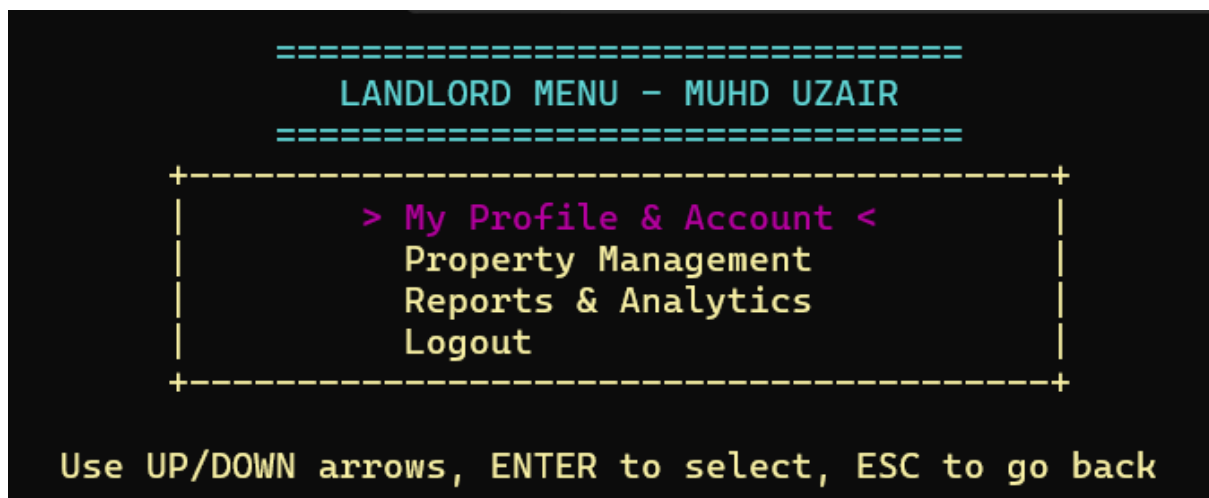


Figure 3.6.1: Landlord Main Menu Interface



Figure 3.6.2: Representative Main Menu Interface

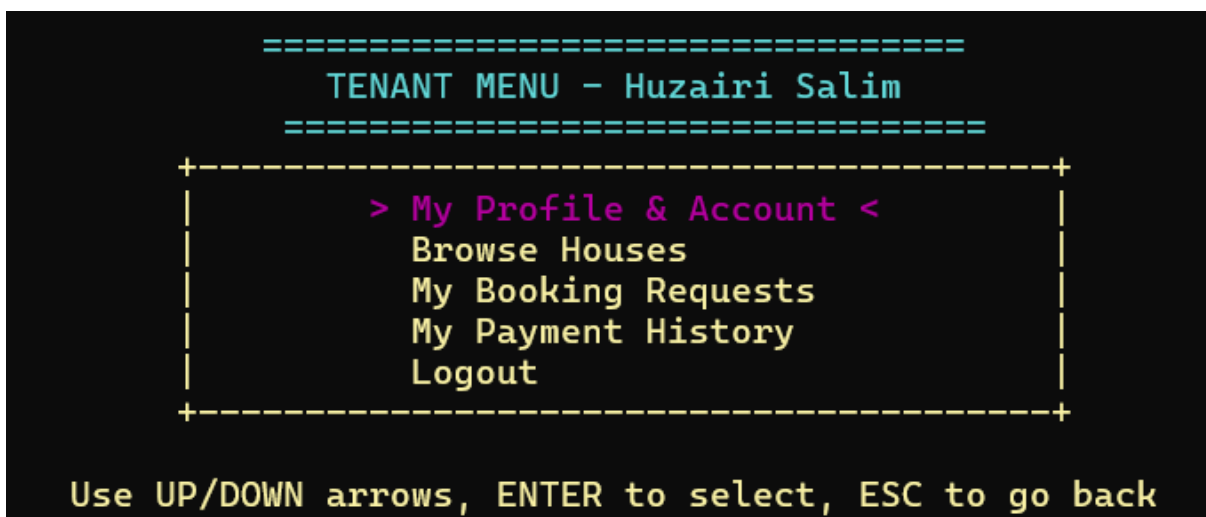


Figure 3.6.3: Tenant Main Menu Interface

```
+-----+
Use UP/DOWN arrows, ENTER to select, ESC to go back

=== [LOGIN AS 'Landlord'] ===

Username: ZAI
Password: 123

--- Login successful. Welcome, Muhd Uzair! ---
```

Figure 3.6.4: Example of Successful message feedback

```
=== [EDIT NAME] ===

Enter new name:

--- name cannot be empty. ---
```

Figure 3.6.4: Example of Error message feedback

CHAPTER 4: IMPLEMENTATION

4.1 Naming Convention

The project follows a generally consistent naming convention, which aids readability.

- **Functions:** **camelCase** is used for function names (e.g., `addHouse`, `viewTenantProfile`, `dbConnect`).
- **Variables:** **camelCase** is also used for local variables and struct members (e.g., `landlordId`, `houseAddress`, `newHouse`).
- **Database Columns:** **PascalCase** is used for database column names (e.g., `LandlordID`, `HouseID`, `PaymentDate`).
- **Macros/Constants:** **UPPER_SNAKE_CASE** is used for constants and macros (e.g., `KEY_UP`, `COL_TITLE`).

Table 4.1.1: Naming Convention Examples

Category	Convention	Example
Functions	camelCase	<code>generatePaymentReceipt()</code>
Local Variables	camelCase	<code>tenantId</code> , <code>monthlyRent</code>
Struct Members	snake_case	<code>landlord_id</code> , <code>start_date</code>
DB Columns	PascalCase	<code>RepresentativeID</code> , <code>TenancyStatus</code>
Constants	UPPER_SNAKE_CASE	<code>COL_WARN</code> , <code>TOTAL_WIDTH</code>

4.2 Function

Functions are the primary unit of code organization. The implementation shows an approach where functions are grouped by the entity they operate on (e.g., all landlord CRUD functions are together).

```
// =====
// FUNCTION PROTOTYPES
// =====

// Landlord CRUD
void addLandlord(sql::Connection* con, const Landlord& l);
void viewLandlordProfile(sql::Connection* con, int landlordId);
void listLandlords(sql::Connection* con);

// Representative CRUD
void addRepresentative(sql::Connection* con, const Representative& r);
void viewRepresentativeProfile(sql::Connection* con, int repId);
void listRepresentatives(sql::Connection* con);

// Tenant CRUD
void addTenant(sql::Connection* con, const Tenant& t);
void viewTenantProfile(sql::Connection* con, int tenantId);
void listTenants(sql::Connection* con);
void autoUpdateTenancyStatuses(); // runs periodically to update statuses
void moveOutTenantEarly(int tenantId); // action by tenant
void cancelTenancy(int tenantId); // administrative action

// House CRUD
void addHouse(sql::Connection* con, const House& h);
void updateHouse(sql::Connection* con, const House& h);
void updateHouseOccupancy(sql::Connection* con, int houseId); // recompute cu
int getAvailableSlots(sql::Connection* con, int houseId); // get available sl
void listHouses(sql::Connection* con, int landlordId); // for landlord view
void listAssignedHouses(sql::Connection* con, int repId); // Houses assigned
void listTenantsInAssignedHouses(sql::Connection* con, int repId); // Tenants
void viewHouse(sql::Connection* con, int houseId); // detailed view for ONE h
void deleteHouse(sql::Connection* con, int houseId, int requesterLandlordId);
```

Figure 4.2.1: Example of Function Prototypes in db_cruds.h

```
// Report Generation
void generateLandlordRevenueSummary(sql::Connection* con, int landlordId);
void generateLandlordOutstandingReport(sql::Connection* con, int landlordId);
void generateLandlordOccupancyReport(sql::Connection* con, int landlordId);
void generateLandlordVacancyReport(sql::Connection* con, int landlordId);
void exportLandlordFullReport(sql::Connection* con, int landlordId);
void generateRepresentativePaymentReport(sql::Connection* con, int repId);

// Receipt Generation
void generatePaymentReceipt(sql::Connection* con, int paymentId);
void generateBatchReceipts(sql::Connection* con, int repId, const string& startDate, const string& endDate);

// User Management
void updateUserPassword(sql::Connection* con, const string& userType, int userId, const string& password);
bool verifyUserPassword(sql::Connection* con, const string& userType, int userId, const string& password);
void changePassword(sql::Connection* con, const string& role, int userId);
void deleteAccount(sql::Connection* con, const string& userType, int userId);
void updateUserField(void* con, const string& role, int userId, const string& field, const string& value);
void updateProfileField(void* con, const string& role, int userId, const string& field);
void profileMenu(sql::Connection* con, const string& role, int userId);
```

Figure 4.2.2: Example of Function Prototypes in misc_functions.h

```
#include <cppconn/exception.h>

using namespace std;

// =====
//  FUNCTION PROTOTYPES
// =====

inline int arrowMenu(const vector<string>& options);
inline void printColoredLn(const string& s, WORD color);
inline void printPaddedPrompt(const string& prompt);
inline void printInfoLn(const string& text);
inline void printCerrColored(const string& s);
inline void printSuccessLn(const string& text);
inline void printWarningLn(const string& text);
inline void initConsoleColors();
```

Figure 4.2.3: Example of Function Prototypes in ui_helpers.h

- **db_cruds.h**: Contains the core business logic functions like `addLandlord`, `reviewBookingRequest`, and `listHouses`. These functions often take a `sql::Connection*` as the first argument, indicating they perform database operations.
- **misc_functions.h**: Contains utility functions for more complex operations that span multiple entities, such as `generateLandlordRevenueSummary` and `changePassword`.
- **ui_helpers.h**: Contains functions purely for presentation, such as `arrowMenu` and `printCentered`.
- **Main.cpp**: Contains the menu functions (`LandlordMenu`, `TenantMenu`) that control the application flow.

4.3 Array

The `std::vector` is the primary array-like data structure used throughout the project, which is the modern C++ best practice as of today.

```
inline int arrowMenu(const vector<string>& options, c
    int startY = -1)
{
    if (options.empty()) return -1;

```

Figure 4.3.1: Example of std::vector use in arrowMenu() function

```
printCentered( notes. + (req.notes

vector<string> reviewOptions = {
    "Approve Request",
    "Reject Request",
    "Cancel"
};

```

Figure 4.3.1: Example of std::vector use in reviewPendingRequestsUI() function

- **Menu Options:** `std::vector<string>` is used in `arrowMenu` to hold the list of options to be displayed to the user (e.g., `mainOptions` in `LandlordMenu`).
- **Data Caching:** In functions like `reviewPendingRequestsUI` and `searchBookHouse`, results from a database query are temporarily stored in a `std::vector` of structs (e.g., `vector<ReqRow> list`). This allows the system to display the data and then refer back to it by index when the user makes a selection, avoiding a second database query.

4.4 Selection

Selection logic is implemented using `if-else` statements and `switch` cases.

```
switch (choice)
{
    case 0: // Profile & Account
    {
        auto con = dbConnect();
        bool backToMain = false;

        while (!backToMain)
        {
            system("cls");
            string headerText = "=== [MY PROFILE & ACCOUNT] ===";
            string line = string(headerText.length(), '=');

            setColor(COL_TITLE);
            printCentered(line + "\n");
            printCentered(headerText + "\n");
            printCentered(line);
            resetColor();

            vector<string> profileOptions = {
                "View My Profile",
                "Manage My Profile (Edit Details)",
                "Delete My Account",
                "Back to Main Menu"
            };
        }
    }
}
```

Figure 4.4.1: Example of switch case in `LandlordMenu()` function

```
stmt = stmt1, houseId);
int rows = stmt->executeUpdate();
if (rows > 0)
    printSuccessLn("House deleted successfully.");
else
    printWarningLn("No house found with that ID.");
```

Figure 4.4.1: Example of if-else statement in `deleteHouse()` function

- **switch cases:** Used extensively in the main menu functions (`LandlordMenu`, `TenantMenu`, etc.) to handle user choices from the `arrowMenu` function. This is an efficient and readable way to branch logic based on a user's selection.
- **if-else statements:** Used for binary decisions and input validation. For example, in `Login`, an `if-else` block checks if the database password matches the with the provided password. In `deleteHouse`, an `if` statement checks if the

requester has permission to delete the property.

4.5 Control

Control flow is managed by loops and conditional statements.

```
// ----- LANDLORD MENU -----
void LandlordMenu(int landlordId, const str
{
    do
    {
        system("cls");
        setColor(COL_TITLE);

        const size_t MAX_NAME_LEN = 30;
        string displayName = truncateStr(lan
        string headerText = "LANDLORD MENU -
        string line = string(headerText.leng
```

Figure 4.5.1: Example of do-while loops in LandlordMenu() function

```
while (res->next()) {
    cout << pad << left
        << setw(NAME_W) << tru
        << setw(PHONE_W) << tr
        << setw(USERNAME_W) <<
        << setw(START_W) << tr
        << "\n";
}
```

Figure 4.5.2: Example of while loops in listTenants() function

- **do-while loops:** Form the backbone of the menu system. Each menu function (e.g., LandlordMenu) runs inside a do-while(true) loop, continuously displaying the menu after an action is completed. The loop is only broken when the user explicitly chooses to "Logout" or "Back," at which point a return statement is executed.
- **while loops:** Used for iterating through database ResultSet objects (e.g., while (res->next())). This is the standard use case for processing rows returned from a MySQL query.

4.6 Pointer

Pointers are used for managing database resources and are essential for the MySQL Connector/C++ library.

```
if (houseId > 0) {
    query += " WHERE h.HouseID = ?";
    unique_ptr<sql::PreparedStatement> stmt(con->prepareStatement(query))
    stmt->setInt(1, houseId);
    stmt->executeUpdate();
} else {
    unique_ptr<sql::Statement> stmt(con->createStatement());
    stmt->execute(query);
}
```

Figure 4.6.1: Example of `unique_ptr` in `updateHouseOccupancy()` function

```
// Update occupancy for a single house (or all if houseId = -1)
void updateHouseOccupancy(sql::Connection* con, int houseId = -1) {
    try {
        string query = "UPDATE house SET occupancy = 0 WHERE houseId = ?";
```

Figure 4.6.2: Example of `sql::Connection* con`

- **`std::unique_ptr`**: This pointer is used for almost all database objects (Connection, PreparedStatement, ResultSet). This implementation choice ensures that the database resources are automatically released when the pointer goes out of scope, preventing memory leaks.
- **Raw Pointers**: Raw pointers like `sql::Connection* con` are used to pass the database connection object between functions.

4.7 Error Handling

Error handling is primarily accomplished through `try-catch` blocks for database exceptions.

```

try {
    string query = "UPDATE payment SET Amount = ?, Notes = ? WHERE Pa
    unique_ptr<sql::PreparedStatement> stmt(con->prepareStatement(que
    stmt->setDouble(1, newAmount);
    stmt->setString(2, newNote);
    stmt->setInt(3, paymentId);
    int rows = stmt->executeUpdate();
    if (rows > 0)
        printSuccessLn("Payment updated successfully.");
    else
        printWarningLn("Payment not found.");
}
catch (sql::SQLException& e) {
    printCerrColored(string("SQL Error (editPayment): ") + e.what());
}

```

Figure 4.7.1: Example of try-catch for database error handling

```

if (list.empty()) {
    printCentered("No houses found for you.\n");
    return -1;
}

```

Figure 4.7.2: Example of error checking for empty input

```

// Validate month format
if (!isValidMonth(startMonth)) {
    printWarningLn("Invalid month format. Use YYYY-MM.");
    system("pause");
    delete con;
    return;
}

// Validate start month is not in the past
if (startMonth < currentMonthStr) {
    printWarningLn("Start month cannot be in the past.");
    system("pause");
    delete con;
    return;
}

```

Figure 4.7.3: Example of validating input for month in searchBookHouse() function

```

if (!checkRes->next()) {
    printWarningLn("No house found with that ID.");
    return;
}

int ownerId = checkRes->getInt("LandlordID");
if (ownerId != requesterLandlordId) {
    printWarningLn("Permission denied: You can only delete your own houses.");
    return;
}

```

Figure 4.7.4: Example of permission verification before execution

- **try-catch** (`sql::SQLException& e`): Most functions that interact with the database are wrapped in this block. If a database error occurs (e.g., connection lost, syntax error in query), the exception is caught by the block.
- **Error Reporting**: Inside the `catch` block, the error message (`e.what()`) is printed to the console using a custom colored error function (`printCerrColored`). This provides immediate, visible feedback to the user or developer about the database issue.
- **Input Validation**: The code also performs logical error handling, such as checking for empty input, validating selections are within a valid range, and verifying user permissions before executing an action.

4.8 Business Rule Implementation

The business rules are implemented directly within the C++ functions.

```
// Check vacancy limit
if (getAvailableSlots(con, houseId) <= 0) {
    unique_ptr<sql::PreparedStatement> upd(con->prepareStatement(
        "UPDATE booking_request SET Status = 'Rejected', Notes = COALESCE(CO
    ));
    upd->setInt(1, bookingId);
    upd->executeUpdate();
    printWarningLn("Cannot approve: house has reached its vacancy limit.");
    return;
}
```

Figure 4.8.1: Example of vacancy check before tenant can book a house

```
// If a landlord id is supplied, verify ownership before deleting.
if (requesterLandlordId > 0) {
    unique_ptr<sql::PreparedStatement> checkStmt(con->prepareStatement(
        "SELECT LandlordID FROM house WHERE HouseID = ?"
    ));
    checkStmt->setInt(1, houseId);
    unique_ptr<sql::ResultSet> checkRes(checkStmt->executeQuery());

    if (!checkRes->next()) {
        printWarningLn("No house found with that ID.");
        return;
    }

    int ownerId = checkRes->getInt("LandlordID");
    if (ownerId != requesterLandlordId) {
        printWarningLn("Permission denied: You can only delete your own houses
    }
}
```

Figure 4.8.2: Example of permission check for destructive actions

- Example: **Vacancy Check** In `reviewBookingRequest`, before approving a booking, the code calls `getAvailableSlots(con, houseId)`. If the result is less than or equal to zero, the request is automatically rejected, and a note is added. **This enforces the rule that a house cannot be overbooked.**
- Example: **Permission Check** In `deleteHouse`, if a `requesterLandlordId` is provided, the function first queries the database to find the `LandlordID` associated with the `houseId`. It then compares the two IDs. If they do not match, it prints a "Permission denied" error and returns, **enforcing the rule that landlords can only delete their own properties.**

4.9 Complex Calculation Implementation

Complex calculations are implemented in `misc_functions.h` and `db_cruds.h` to support financial analysis.

```
// Get total paid
string query = R"(
    SELECT IFNULL(SUM(Amount), 0) AS total_paid
    FROM payment
    WHERE TenantID = ? AND HouseID = ?
    AND Status IN ('Paid', 'Partial')
)";

unique_ptr<sql::PreparedStatement> stmt(con->prepareStatement(query));
stmt->setInt(1, tenantId);
stmt->setInt(2, houseId);
unique_ptr<sql::ResultSet> res(stmt->executeQuery());

if (res->next()) {
    double totalPaid = res->getDouble("total_paid");
    double totalExpected = info.monthlyRent * info.monthsElapsed;
    double balance = totalPaid - totalExpected;
    return balance > 0 ? balance : 0; // Only return positive credit
}
```

Figure 4.9.1: Calculation for overpaid and forwarding to following month

```
// Get credit balance from previous months
status.creditBalance = getTenantCreditBalance(con, tenantId, houseId);

// Calculate remaining due and overpayment
double effectivePayment = status.totalPaidThisMonth + status.creditBalance;

if (effectivePayment >= status.monthlyRent) {
    status.isFullyPaid = true;
    status.remainingDue = 0;
    status.overpayment = effectivePayment - status.monthlyRent;
}
else {
    status.isFullyPaid = false;
    status.remainingDue = status.monthlyRent - effectivePayment;
    status.overpayment = 0;
}
```

Figure 4.9.2: Complex calculation of tenant's payment status

```
void generateLandlordRevenueSummary(sql::Connection* con, int landlordId) {
    try {
        string query = R"(
            SELECT
                h.HouseID,
                h.Address,
                h.Rent AS monthly_rent,
                h.VacancyLimit,
                h.CurrentOccupancy,
                COUNT(DISTINCT t.TenantID) AS tenant_count,
                COUNT(p.PaymentID) AS payment_count,
                IFNULL(SUM(CASE WHEN p.Status = 'Paid' THEN p.Amount ELSE 0 END), 0) AS total_paid,
                IFNULL(SUM(CASE WHEN p.Status = 'Overdue' THEN p.Amount ELSE 0 END), 0) AS total_overdue,
                IFNULL(SUM(p.Amount), 0) AS total_expected
            FROM house h
            LEFT JOIN tenant t ON t.HouseID = h.HouseID AND t.TenancyStatus = 'Active'
            LEFT JOIN payment p ON p.HouseID = h.HouseID
            WHERE h.LandlordID = ?
            GROUP BY h.HouseID, h.Address, h.Rent, h.VacancyLimit, h.CurrentOccupancy
            ORDER BY total_received DESC
        )";
```

Figure 4.9.3: Example of complex SQL query for financial data of every property

- `getTenantCreditBalance`: This function calculates how much a tenant has overpaid. It first determines the total amount of rent expected since the tenancy began (`info.monthlyRent * info.monthsElapsed`) and then subtracts this from the total amount the tenant has actually paid.
- `getTenantPaymentStatus`: This function provides a full snapshot of a tenant's status for the current month. It combines the `monthlyRent`, the `totalPaidThisMonth`, and the `creditBalance` from previous months to determine if the rent is fully paid and calculates any `remainingDue` or `overpayment`.
- `generateLandlordRevenueSummary`: This function uses a complex SQL

query with SUM, CASE, and GROUP BY to aggregate financial data of every property. It calculates `total_received` and `total_overdue` and then computes summary statistics like `collectionRate` and `occupancyRate` in the system.

4.10 Report Generation for Analysis Implementation

Report generation is a key feature, implemented in `misc_functions.h`.

```

=====
=== [FINANCIAL SUMMARY REPORT] ===
=====

=====
REVENUE SUMMARY BY PROPERTY
=====
Property Address      Monthly RentOccupancy  Received  Overdue  Expected
=====
Penang Georgetown House RM 110      1/4      RM 2710  RM 0     RM 2710
Wangsa Maju Apartment  RM 130      1/3      RM 1490  RM 0     RM 1590
=====

Summary Statistics:
- Total Properties: 2
- Total Slots: 7 (Occupied: 2, Vacant: 5)
- Occupancy Rate: 28.6%
- Collection Rate: 97.7%
- Net Revenue: RM 4200.00

=====
OUTSTANDING PAYMENTS REPORT
=====
Property      Tenant      Phone      Amount      Days Late
=====

--- No outstanding payments! All tenants are up to date. ---

```

Figure 4.10.1: Example output for financial summary report

```

=====
=== [PROPERTY OVERVIEW REPORT] ===
=====

```

CURRENT TENANTS & OCCUPANCY STATUS					
Property Address	Slots	Tenant Name	Phone	Start Date	Status
Penang Georgetown House	1/4	Priya Devi	019-8901234	2024-05-01	Active
Wangsa Maju Apartment ..	1/3	Linda Tan	012-0123456	2024-06-01	Active

```

=====
Occupancy Statistics:
- Total Slots: 7
- Occupied Slots: 2
- Available Slots: 5
- Occupancy Rate: 28.6%
=====

```

VACANCY ANALYSIS REPORT					
Property	Rent/Month	Avail/Total	Deposit	Representative	Contact
Penang Georgetown House	RM 110	3/4	RM 220	RJ Zaniel	011-3333333
Wangsa Maju Apartment ..	RM 130	2/3	RM 260	Zuhairi	011-1111111

```

=====
Vacancy Impact:
- Properties with Vacancies: 2
- Total Vacant Slots: 5
- Potential Monthly Revenue Loss: RM 590.00
- Annual Impact: RM 7080.00
=====

```

Figure 4.10.2: Example output for property overview report

```

=====
=== [EXPORT COMPLETE REPORT TO FILE] ===
=====

--- Full report exported successfully: Landlord_Report_20260112_203245.txt ---

Report saved to: Landlord_Report_20260112_203245.txt

```

```

=====
COMPREHENSIVE LANDLORD REPORT
=====

Generated: 20260112_203245
Landlord ID: 2

=== REVENUE SUMMARY ===

=====
REVENUE SUMMARY BY PROPERTY
=====

Property Address      Monthly RentOccupancy Received   Overdue   Expected
-----
Penang Georgetown House RM 110    1/4      RM 2710   RM 0      RM 2710
Wangsa Maju Apartment .. RM 130    1/3      RM 1490   RM 0      RM 1590
=====

Summary Statistics:|
- Total Properties: 2
- Total Slots: 7 (Occupied: 2, Vacant: 5)
- Occupancy Rate: 28.6%
- Collection Rate: 97.7%
- Net Revenue: RM 4200.00

=== OUTSTANDING PAYMENTS ===

=====
OUTSTANDING PAYMENTS REPORT
=====

Property      Tenant      Phone      Amount      Days Late
-----

--- No outstanding payments! All tenants are up to date. ---

Ln 22, Col 28   3,980 characters   Plain text   100%   Windows (CRLF)   UTF-8

```

Figure 4.10.3: Example of exporting and viewing the full report

- **On-Screen Reports:** Functions like `generateLandlordOccupancyReport` execute a detailed SQL query, iterate through the results, and print a formatted table directly to the console using `setw` and helper functions for alignment and truncation.
- **File Export:** The `exportLandlordFullReport` function demonstrates an advanced implementation of the report system.
 - i. It generates a unique filename using a timestamp (`"Landlord_Report_YYYYMMDD_HHMMSS.txt"`).
 - ii. It opens an `ofstream` with this filename.
 - iii. It temporarily redirects `std::cout` to the file's buffer.
 - iv. It calls the existing on-screen report generation functions, whose output is now written to the file instead of the console.
 - v. Finally, it restores `std::cout`'s original buffer and closes the file.

CHAPTER 5: CONCLUSION

5.1 Constraints

The project was developed under several technical and practical constraints:

- **Console Based Interface:** The system is limited to a text-based user interface, which restricts the data visualization and user interaction compared to a graphical or web-based application.
- **MySQL Dependency:** The system is tightly integrated to the MySQL database via the specific MySQL Connector/C++ library. Migrating to another database system would require significant code changes.
- **Security Implementation:** The lack of secure password hashing and the presence of hardcoded credentials make the implementation focused on functionality over security, making it unsuitable for deployment in a production environment without major refactoring.

5.2 Future Improvements

The HRMS provides a solid foundation that can be extended with numerous improvements:

1. **Critical Security Overhaul:**
 - a. **Password Hashing:** Implement a strong, hashing algorithm for storing all user passwords.
 - b. **Externalize Credentials:** Move all database credentials from the source code to a separate, secure configuration file that is not tracked.
2. **Transition to a GUI or Web Interface:** Rebuild the presentation layer using a modern framework to provide a more intuitive and richer user experience.
3. **Advanced Reporting and Dashboards:** Add more advanced reporting features, such as graphical charts for revenue trends, occupancy history, and interactive data filtering.
4. **Better Filtering and Searching for Booking:** To improve the booking process, we will implement an advanced search that allows tenants to filter properties by

combined criteria like price, location, house type, and specific date availability for a more precise user experience.

5. **Data Archiving:** Develop a mechanism to archive old records such as completed tenancies or old payments, to keep the primary database tables clean.
6. **Robust Input Checker:** Although the current system already applied advanced input checker and error, some inputs can still cause misbehave to the system.
7. **Better UI/UX implementation:** Implement a better user experience by reducing deep menus and submenus, better arrangement for table outputs and implement consistent design and colour throughout the system's interface.

REFERENCES

1. MySQL Connector/C++ 8.0 Developer Guide. (n.d.). Retrieved from <https://dev.mysql.com/doc/connector-cpp/8.0/en/>
2. Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley Professional.
3. The C++ Core Guidelines. Retrieved from <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>
4. C++ Tutorial. (n.d.). Retrieved from [W3Schools.com](https://www.w3schools.com/cpp/).
<https://www.w3schools.com/cpp/>
5. MySQL Tutorial. Retrieved from [W3Schools.com](https://www.w3schools.com/MySQL/default.asp).
<https://www.w3schools.com/MySQL/default.asp>